

(1) 有三个进程 R、W1、W2 共享一个缓冲器 B，B 中每次只能存放一个数。进程 R 每次启动输入设备读一个数且把它存放到缓冲器 B 中，若存放到缓冲器中的是奇数，则由 W1 将其取出打印；若存放到缓冲器中的是偶数，则由 W2 将其取出打印。要求用 PV 操作管理这三个并发进程，使它们能正确地同步工作。

```
semaphore s_empty=1,s_even=0,s_odd=0;
```

```
item buff;
```

```
void r()
```

```
{L1:
```

```
    prepare an number x;
```

```
    wait(s_empty);
```

```
    put x in buff;
```

```
    if(x%2) signal(s_odd); //若 x 为奇数
```

```
    else signal(s_even);
```

```
    goto L1;
```

```
}
```

```
void w1()
```

```
{L2:
```

```
    wait(s_odd);
```

```
    get the number;
```

```
    signal(s_empty);
```

```
    Print the number;
```

```
    goto L2;
```

```
}
```

```
void w2()
```

```
{L3:
```

```
    wait(s_even);
```

```
    get the number;
```

```
    signal(s_empty);
```

```
    print the number;
```

```
    goto L3;
```

```
}
```

(2) 桌上有一空盘，允许存放一个水果。爸爸可向盘中放苹果，妈妈可向盘中放桔子。女儿专等吃盘中的苹果，儿子专等吃盘中的桔子。只要盘子空爸爸或妈妈可向盘中放一个水果。用 PV 操作进行管理使他们能正确地执行。【参考课件】

(3) 某公司由生产车间和销售车间组成，生产车间生产产品并将产品放入仓库；销售车间从仓库中取出产品进行销售。若仓库中能存放 100 个产品，并且只有一台运输车辆供生产部门和销售部门运送货物，车上只能放一个产品，请使用 wait 和 signal 原语描述上述过程。

```
item buffer[100];
```

```
semaphore item=0, position=100; //消息信号量
```

```
semaphore mutex_car=1;
```

```
int in=0,out=0;
```

```
void produce()
```

```
{
```

```
    while(1)
```

```
    { produce an item x;
```

```
      wait(position);
```

```
      wait(mutex_car);
```

```

        put x on the car;
        buffer[in]=x;
        in=(in+1)%100;
        signal(mutex_car);
        signal(item);
    } //end while
}
void saler()
{
    while(1)
    { wait(item);
      wait(mutex_car);
      y=buffer[out];
      out=(out+1)%100;
      signal(mutex_car);
      signal(position);
      sale item y;
    } //end while
}

```

(4) 某工厂有两个生产车间和一个装配车间，两个生产车间分别生产 A、B 两种零件，装配车间的任务是将 A、B 组合到一起成为产品。两个车间每生产一个零件后，都要把它们送到装配车间的货架 F1、F2 上。F1 存放零件 A，F2 存放零件 B，F1 和 F2 的容量均存放 10 个零件。装配工人每次从货架取一个零件 A 和一个零件 B 后组装成产品。请用 wait、signal 操作进行正确管理。

```

semaphore mutex1=1;//A 车间与装配车间互斥使用 F1
semaphore mutex2=1;//B 车间与装配车间互斥使用 F2
int in1=int2=out1=out2=0;//F1,F2 的读写指针
void A()//生产车间 A
{
    while(1)
    {
        produce an item a;
        wait(position1);//申请 F1 的位置
        wait(mutex1);//申请互斥使用 F1
        F1[in1]=a;
        in1=(in1+1)%n;
        signal(mutex1);
        signal(item1);
    }
}
void B()//生产车间 B
{

```

```

        while(1)
        {
            produce an item b;
            wait(position2);//申请 F2 的位置
            wait(mutex2);//申请互斥使用 F2
            F2[in2]=b;
            in2=(in2+1)%n;
            signal(mutex2);
            signal(item2);
        }
    }
    void ASSEMBL()//组装车间
    {
        while(1)
        {
            wait(item1);//申请产品 A
            wait(mutex1);//申请互斥使用 F1
            x=F1[out1];
            out1=(out1+1)%n;
            signal(mutex1);
            signal(positoin1);
            wait(item2);//申请产品 B
            wait(mutex2);//申请互斥使用 F2
            y=F2[out2];out2=(out2+1)%n;
            signal(mutex2);
            signal(positoin2);
            Assemble x and y;
        }
    }
}

```

(5) 某小型超级市场，可容纳 50 人同时购物。入口处备有篮子，每个购物者从入口入内购物，在出口处结帐（出口和入口禁止多人同时通过）。试用 wait(s)、signal(s)操作为同步机制写出它们并发执行时能正确工作的程序。

```

semaphore sm=50;//控制并发度的信号量
semaphore in_mut=1,out_mut=1;//入口、出口互斥使用
void consumeri()
{L1:    wait(sm);//是否超市中有位置
        wait(in_mut);//申请入口
        通过入口;
        signal(in_mut);//释放入口
        在超市中购物;
        wait(out_mut);//申请出口
        通过出口;

```

```

        signal(out_mut);//释放出口
        signal(sm);//释放位置
        goto L1;
    }
}

```

(6) 有 $n(n \geq 3)$ 名哲学家围坐在一张圆桌边，每名哲学家要进行就餐和思考。在圆桌中心有 $m(m > 1)$ 个碗，每两名哲学家之间有一根筷子，即 n 个筷子。每名哲学家必须取到一个碗和两侧的筷子后，才能就餐，就餐完毕，将碗和筷子放回原处，继续思考。为使尽可能多的哲学家同时进餐，且防止出现死锁现象，请使用信号量的 PV 操作描述上述过程中的同步与互斥。

【提示】分 $m < n$ 和 $m \geq n$ 两种情况考虑。若 $m < n$ 则系统并发度为 m ，若 $m \geq n$ 则系统并发度控制为 $n-1$ 。

```

semaphore bowl;//因为碗也是互斥资源，所以用碗的数量控制系统并发度
semaphore ck[n];//n 个筷子，n 个互斥资源
void p(int i)
{
    //信号量的初始化
    for(j=0;j<n;j++) ch[j]=1;
    bowl=min{m,n-1};//碗数 m 少于人数 n，则 bowl 取 m;碗数 m 多于人数 n，则 bowl
                        取 n-1。保证系统中最多允许 n-1 个哲学家进餐。
    while(1)
    {
        wait(bowl);//申请碗（位置）
        wait(ch[i]);
        wait(ch[(i+1)%n]);
        eat;
        signal(ch[i]);
        signal(ch[(i+1)%n]);
        think;
    }
}
}

```

(7) 吸烟者问题：假设一个系统有 3 个抽烟者进程和一个供应者进程。

每个抽烟者进程不停地卷烟并抽掉它，但要卷起并抽掉一只烟，抽烟者需要三种材料：烟草、纸、胶水。

三个抽烟者中，第一个有烟草，第二个有纸，第三个有胶水。

供应者进程无限地供应三种原料，供应者每次随机将两种材料放到桌子上，并通知缺这两种材料的抽烟者。

相应抽烟者取走供应的材料后卷烟并抽掉，并通知供应商已完成，供应商再次随机供应两种原材料。

请使用 PV 操作实现上述同步。

提示：供应者进程根据随机数投放原料、发出通知。

```

semaphore offer1=0;//提供烟草和纸的信号量;
semaphore offer2=0;//提供烟草和胶水的信号量;
semaphore offer3=0;//提供纸和胶水的信号量;
semaphore finish=0;//取走原料的信号量，桌子空
void p()//供应者进程

```

```

{
    while(1)
    {
        radm=radom();//生成随机数
        radm=radm%3;//随机数在 0-2 之间
        wait(finish);
        if(radm==0)//提供烟草和纸
        {
            放烟草和纸;
            signal(offer1);
        }
        else if(radm==1)//提供烟草和胶水
        {
            放烟草和胶水;
            signal(offer2);
        }
        else
        {
            放纸和胶水;
            signal(offer3);
        }
    } //end while
} // end P

```

```

void c1()//抽烟者 1，有烟草，缺纸和胶水
{
    while(1)
    {
        wait(offer3);
        取走纸和胶水;
        signal(finish);
        卷烟;
        抽烟;
    }
}

```

```

void c2()//抽烟者 2，有纸，缺烟草和胶水
{
    while(1)

```

```

        {
            wait(offer2);
            取走烟草和胶水;
            signal(finish);
            卷烟;
            抽烟;
        }
    }

void c3()//抽烟者 3, 有胶水, 缺烟草和纸
{
    while(1)
    {
        wait(offer1);
        取走烟草和纸;
        signal(finish);
        卷烟;
        抽烟;
    }
}

```

(7) 对于读者—写者问题（读者优先）算法修改，实现只允许 8 个读者同时读文件 F。试用 wait(s)、signal(s)操作作为同步机制写出它们并发执行时能正确工作的程序。

```

int readcount=0;
semaphore fmutex=1;//文件 F 的互斥信号量
semaphore rmutex=1;//readcount 的互斥信号量
semaphore sm=8;//控制系统的最大并发度为 8，即只允许 8 个读者
void writer()//i=1,2,3,……
{
    while(1) {
        wait(fmutex);
        write file;
        signal(fmutex);
    }
}
void reader()//j=1,2,3,……
{
    while(1){
        wait(sm);//检测读者人数是否达到 8
        wait(rmutex);
        if(readcount==0) wait(fmutex);
        readcount++;
        signal(rmutex);
    }
}

```

```

        read file;

        wait(rmutex);
        readcount--;
        if(readcount==0) wait(fmutex);
        signal(rmutex);
        signal(sm); //释放一个读者位置
    }
}

```

(8) 有两组读进程共享文件 F，现规定：同一组的读进程可以同时读共享文件 F，不同组的进程互斥访问共享文件 F。试用 wait(s)、signal(s) 操作作为同步机制写出它们并发执行时能正确工作的程序。

```

int readcount1=0; //第一组的读者人数
int readcount2=0; //第二组的读者人数
semaphore fmutex=1; //文件 F 的互斥信号量
semaphore rmutex1=1; //readcount1 的互斥信号量
semaphore rmutex2=1; //readcount2 的互斥信号量
void reader1() //第一组读者进程
{
    while(1){
        wait(rmutex1);
        if(readcount1==0) wait(fmutex); //第一组的第一个进程申请文件 F
        readcount1++;
        signal(rmutex1);

        read file;

        wait(rmutex1);
        readcount1--;
        if(readcount1==0) wait(fmutex); //第一组的最后一个进程释放文件 F
        signal(rmutex1);
    } //end while
}
void reader2() //第二组读者进程
{
    while(1){
        wait(rmutex2);
        if(readcount2==0) wait(fmutex); //第二组的第一个进程申请文件 F
        readcount2++;
        signal(rmutex2);
    }
}

```

```

        read file;

        wait(rmutex2);
        readcount2--;
        if(readcount2==0) wait(fmutex); //第二组的最后一个进程释放文件
        signal(rmutex2);
    } //end while
}

```

(10) 根据对写者优先的分析，试用 wait(s)、signal(s)操作为同步机制写出实现写者优先的进程描述。

```

int readcount=0; //文件 F 中的读者人数
int writecount=0; //VIP 中等候访问文件 F 的写者人数
semaphore rmutex=1; //readcount 的互斥
semaphore wmutex=1; //writecount 的互斥
semaphore fmutex=1; //文件 F 互斥
semaphore viproom=1; //VIP 的互斥
void writer()
{
    while(1){
        //进入 VIP
        wait(wmutex); //writecount 为多个写者共享变量，需要互斥访问
        if(writecount==0) wait(viproom); //第一个写者申请 VIP，其他写者进入 VIP
        writecount++;
        signal(wmutex);

        //访问文件
        wait(fmutex); //申请使用文件
        write file;
        signal(fmutex); //释放文件 F

        //退出 VIP
        wait(wmutex);
        writecount--;
        if(writecount==0) signal(viproom); //最后一个写者释放 VIP
        signal(wmutex);
    } //end while
}

void readerj()
{
    while(1){
        wait(viproom); //申请通过 VIP

```



```
wait(rmutex);
if(readcount==0) wait(fmutex);
readcount++;
signal(rmutex);
signal(viproom);//释放 VIP

read file;

wait(rmutex);
readcount--;
if(readcount==0) wait(fmutex);
signal(rmutex);
} //end while
}
```